# Plan de desarrollo — Práctica 002

## SVM de regresión (SVR) para predecir ventas diarias

| Campo | Valor |
|---|---|
| **Asignatura** | Aprendizaje Automático |
| **Práctica** | SVM de regresión (SVR) — desarrollo, optimización y predicción |
| **Equipo** | 4 integrantes |
| **Duración estimada** | 2 semanas, 5 sesiones de trabajo + 1 sesión de buffer |
| **Repositorio** | `tareas/practica-002` |
| **Dataset** | `tareas/practica-001/datos/ventas_features.csv` |

---

## 1. Resumen ejecutivo

El objetivo de esta práctica es desarrollar, optimizar, evaluar y desplegar un modelo de **Máquinas de Soporte Vectorial (SVM)**. La guía permite explícitamente problemas de **regresión y/o clasificación**. Como la variable de interés del dataset es `cantidad_vendida` (continua, en unidades por día), la solución técnicamente correcta es **SVR (Support Vector Regression)** y no una clasificación artificial.

El equipo construirá un único notebook reproducible que:

1. Carga y prepara el dataset de la práctica 001.
2. Entrena SVR con distintos **kernels** ( `linear` , `rbf` , `poly` , `sigmoid` ) y distintos valores de **penalización** `C` .
3. Registra **cada entrenamiento** como una **versión** en el Model Registry de MLflow y consolida todas las versiones en una lista/tabla comparativa.
4. Optimiza el modelo con `GridSearchCV` (C, epsilon, gamma, degree).
5. Define una función `predecir_venta()` para un sample nuevo, probada con **dos casos** reales del dataset (demanda alta y demanda baja).
6. Cierra con conclusiones basadas en métricas y referencias en **APA 7**.

**Resultado esperado (medido previamente sobre este dataset):**

| Modelo | RMSE | MAE | $R^2$ |
|---|---|---|---|
| Baseline ingenuo ( `promedio_movil_7d` ) | 41.72 | 31.42 | — |
| SVR linear, C=1, split temporal | 26.27 | 20.12 | 0.768 |
| SVR RBF, C=10, split aleatorio | 22.42 | 16.68 | 0.848 |
| LSTM de la práctica 001 (referencia) | 23.19 | 17.30 | — |

> Estos valores se midieron durante la planificación con configuraciones distintas a la final (una fila usa split aleatorio y otra selección de columnas); son una referencia orientativa. La tabla consolidada de la sección 8 del notebook, generada desde el registry, es la fuente definitiva.

La conclusión esperada es que un SVR simple alcanza un desempeño comparable al LSTM de la práctica anterior con una fracción de la complejidad.

---

## 2. Objetivo, alcance y exclusiones

### 2.1 Objetivo general

Aplicar SVR al dataset de ventas retail, optimizar sus hiperparámetros, evaluar su desempeño y exponer una función de predicción reutilizable.

### 2.2 Alcance (lo que sí se entrega)

| Ítem | Puntos | Evidencia en el notebook |
|---|---|---|
| 3.1 Desarrollo y optimización de modelos | 2 | Barrido de kernels × C, `GridSearchCV`, tabla y figuras comparativas |
| 3.2 Función de predicción con nuevo sample | 2 | `predecir_venta()` probada con dos casos distintos |
| 3.4 Conclusiones y referencias | 2 | Conclusiones con métricas + referencias APA 7 |

> **Alcance exclusivo:** esta práctica se limita a **SVM/SVR, su optimización y la función de predicción**. No se desarrolla **nada** de **Random Forest** ni **árboles de decisión**, incluido el ejercicio de Information Gain. Tampoco se realiza la **justificación de predicciones con herramientas de explicación** (SHAP, PDP/ICE, permutation importance): ese punto queda explícitamente excluido.

### 2.3 Fuera de alcance

- **Random Forest** y **árboles de decisión** (incluido Information Gain): no se implementan, no se comparan y no forman parte de ningún entregable.
- **Justificación de predicciones** con técnicas de visualización o algoritmos de interpretabilidad (SHAP, PDP/ICE, permutation importance, LIME): no se realiza.
- Despliegue web o API (la práctica 001 ya cubrió eso).
- Entrenamiento en GPU o deep learning.
- Búsqueda exhaustiva de hiperparámetros tipo bayesiano.
- Reentrenamiento con datos nuevos o automatización MLOps.

---

## 3. Decisiones técnicas cerradas

Estas decisiones se toman una vez y no se reabren sin acuerdo del equipo.

| Decisión | Valor | Justificación |
|---|---|---|
| Variable objetivo | `cantidad_vendida` | Es la variable de interés del dataset y evita clases artificiales |
| Tipo de modelo | `SVR` de scikit-learn | SVM para regresión, permitido por la guía |
| Split | Temporal 80/20 por fecha | Evita fuga entre días consecutivos; simula predecir el futuro |
| Escalado de X | `StandardScaler` | SVR es sensible a la escala |
| Escalado de y | `TransformedTargetRegressor` + `StandardScaler` | `epsilon` y `C` dependen de la escala del target |
| Variables categóricas | `categoria` y `product_id` one-hot (25 columnas) | Decisión cerrada del equipo; la duplicación se documenta en la sección 3 del notebook |
| Kernels | `linear`, `rbf`, `poly`, `sigmoid` | Requisito explícito de la guía |
| Penalización C | `0.1, 1, 10, 100` | Requisito explícito; rango suficiente para ver sobreajuste |
| Búsqueda fina | `GridSearchCV` 5-fold sobre C, epsilon, gamma, degree | Optimización de la sección 3.1 |
| Métrica principal | RMSE | Misma unidad que la variable (unidades vendidas) |

| Decisión | Valor | Justificación |
|---|---|---|
| Métricas secundarias | MAE, $R^2$, MAPE | Complementan la interpretación |
| Semilla | `random_state=42` | Reproducibilidad |
| Tracking | MLflow local con backend SQLite (`sqlite:///mlflow.db`) | Requisito de la guía; el registry no funciona con `file:./mlruns` |
| Versiones | Model Registry `svr_ventas`: una versión por kernel × C + la del `GridSearchCV` | Cada combinación es una versión comparable; el alias `champion` marca la vigente |
| Artefactos | `mlartifacts/` (local, ignorado en Git) + `resultados/modelos/*.joblib` (versionado) | El `.db` no siempre viaja; los `.joblib` sí |

### 3.1 Variables de entrada

**Numéricas (11):** `mes`, `dia_del_mes`, `dia_semana`, `es_fin_de_semana`, `es_feriado`, `semana_del_periodo`, `ventas_dia_anterior`, `ventas_hace_7_dias`, `promedio_movil_7d`, `stock_en_momento`, `dias_para_expirar`.

**Categóricas (2, one-hot):** `categoria`, `product_id`.

**Excluidas y por qué:**

| Columna | Motivo |
|---|---|
| `stock_actual` | Constante por producto; identifica al producto, no aporta señal |
| `nombre`, `unidad` | Texto redundante con `product_id` y `categoria` |
| `fecha_venta`, `anio`, `nombre_dia`, `tipo_dia` | Redundantes con las variables de calendario |
| `semana_iso` | No es monótona: el 1-ene-2022 cae en la semana 52 y rompe el orden |

> Las filas con nulos en los rezagos (`ventas_dia_anterior`, `ventas_hace_7_dias`, `promedio_movil_7d`) se eliminan: son 90 celdas en 70 filas del calentamiento de los rezagos (600 → 530 registros). No se imputan porque inventaría ventas que no ocurrieron.

> **Decisión cerrada sobre `categoria`:** entra al modelo junto con `product_id` (25 columnas). Es función de `product_id` (5 de las 6 categorías tienen un solo producto), así que 5 de sus dummies son idénticas a las de `product_id`. Se conserva por acuerdo del equipo y con dos consecuencias conocidas: en el kernel lineal produce colinealidad exacta y en RBF duplica el peso de la identidad del producto en la distancia. La sección 3 del notebook debe explicar la duplicación; `CAT = ["product_id"]` (20 columnas) queda descartado.

> Regla de oro: ninguna variable puede contener información del futuro ni del target. `ventas_dia_anterior` y `promedio_movil_7d` son válidas porque solo usan días previos.

### 3.2 Advertencia sobre `categoria` como target

Se evaluó usar `categoria` como variable objetivo. No se recomienda:

- `Alimentos` concentra 300 filas (5 productos); las otras 5 categorías tienen **1 solo producto** cada una (60 filas).
- Sin variables de fuga, el F1-macro es ≈ 0.30.
- Con `stock_actual` o `dias_para_expirar`, el F1-macro sube a ≈ 0.97, pero es **fuga**: esas columnas son la huella del producto. El modelo identifica productos, no categorías.

Por eso `categoria` se usa como **variable predictora**, no como target.

---

## 4. Arquitectura del notebook

Archivo único: `src/svr_ventas.ipynb`. Regla: cada sección empieza con una celda Markdown que explica qué se hace y por qué.

| # | Sección | Contenido | Rúbrica | Responsable |
|---|---|---|---|---|
| 0 | Portada | Título, integrantes, objetivo, dataset | — | R4 |
| 1 | Configuración | Imports, rutas, semilla, versión de librerías | — | R1 |
| 2 | Datos y EDA mínimo | Carga, nulos, distribución del target, ventas por categoría | — | R1 |
| 3 | Features y split | Selección, one-hot, `dropna`, split temporal | — | R1 |
| 4 | Pipeline SVR | `TransformedTargetRegressor`, `StandardScaler`, `SVR` | 3.1 | R2 |
| 5 | Barrido kernels × C | 16 runs + 16 versiones registradas en MLflow | 3.1 | R2 |
| 6 | Optimización | `GridSearchCV`, mejor modelo, alias `champion` | 3.1 | R2 |
| 7 | Evaluación | RMSE/MAE/$R^2$/MAPE, predicho vs real, residuos | 3.1 | R3 |
| 8 | Resultados consolidados | `search_model_versions` + métricas del run → tabla + CSV + figura | 3.1 | R3 |
| 9 | Función de predicción | `predecir_venta()` + dos casos | 3.2 | R3 |
| 10 | Conclusiones | Hallazgos, comparación con LSTM, limitaciones | 3.4 | R4 |
| 11 | Referencias | APA 7 | 3.4 | R4 |

---

## 5. Equipo, roles y responsabilidades

### 5.1 Perfiles

| Rol | Integrante | Responsabilidad principal | Carga estimada |
|---|---|---|---|
| **R1 — Datos y EDA** | *(nombre)* | Dataset, features, split, control de fuga | 8 h |
| **R2 — Modelado y MLflow** | *(nombre)* | Pipeline SVR, barrido, GridSearch, tracking | 10 h |
| **R3 — Evaluación y predicción** | *(nombre)* | Métricas, figuras, `predecir_venta`, HTML | 9 h |
| **R4 — Conclusiones y documentación** | *(nombre)* | Conclusiones, APA, revisión final | 6 h |

### 5.2 Responsabilidades detalladas

Cada rol tiene un objetivo, tareas verificables, entregables, criterio de aceptación y un límite claro de lo que **no** le corresponde. Las horas son estimadas y suman 33 h de trabajo efectivo para el equipo.

#### 5.2.1 R1 — Datos y EDA (8 h)

**Objetivo:** entregar un dataset limpio, un conjunto de variables sin fuga de información y un split reproducible que el resto del equipo pueda usar sin modificarlo.

**Tareas concretas:**

1. Crear la estructura `datos/`, `resultados/figuras/` y verificar el dataset con `sha256sum -c datos.sha256`.
2. Cargar el CSV con `parse_dates`, ordenar por fecha y producto, y validar dimensiones (600 filas, 10 productos × 60 días), tipos y rangos.
3. Hacer el EDA mínimo: distribución de `cantidad_vendida`, ventas por categoría y producto, evolución temporal y mapa de nulos.
4. Documentar los nulos de `ventas_dia_anterior` (10), `ventas_hace_7_dias` (70) y `promedio_movil_7d` (10), y justificar la decisión de eliminar esas filas (~530 registros útiles).
5. Definir las listas `NUM`, `CAT = ["categoria", "product_id"]` y `TARGET`, y justificar cada exclusión (`stock_actual`, textos, columnas redundantes).

6. Construir `X` con one-hot encoding y `y` con `cantidad_vendida`; verificar que no queden nulos ni columnas constantes.
7. Congelar el split temporal 80/20 por fecha y guardar la lista `COLUMNAS`.
8. Ejecutar una revisión de fuga de información: comprobar que ningún predictor usa datos del futuro ni del target.
9. Escribir en Markdown las secciones 0–3 del notebook y guardar las figuras del EDA.

**Entregables:**

- Notebook secciones 0–3 ejecutando de inicio a fin.
- `resultados/figuras/eda_distribucion.png`, `eda_categoria.png`, `eda_temporal.png`.
- Lista `COLUMNAS` y tabla de variables incluidas/excluidas.
- Documento breve de decisiones de limpieza (puede ser una celda Markdown).

**Criterio de aceptación:**

- `X_train`, `X_test`, `y_train`, `y_test` existen, no tienen nulos y conservan la proporción temporal esperada.
- Cada exclusión de columna está justificada por escrito.
- R2 revisa y aprueba la ausencia de fuga antes de modelar.

**Interacción:** entrega a R2 la matriz y el split; R3 reutiliza las figuras del EDA. **No le corresponde:** entrenar modelos, calcular métricas ni redactar conclusiones.

| Actividad | Horas |
|---|---|
| Estructura y verificación del dataset | 1.0 |
| Carga, validación y EDA | 2.0 |
| Features, one-hot y target | 2.0 |
| Split temporal y revisión de fuga | 1.5 |
| Documentación y figuras | 1.5 |
| **Total** | **8.0** |

### 5.2.2 R2 — Modelado y MLflow (10 h)

**Objetivo:** construir el pipeline SVR, ejecutar el barrido de kernels y penalización, optimizar los hiperparámetros y dejar **cada modelo registrado como versión** en MLflow, de forma reproducible.

**Tareas concretas:**

1. Configurar MLflow con `tracking_uri="sqlite:///mlflow.db"` (el registry necesita un backend SQL) y experimento `svr_ventas`; validar con un run de prueba.
2. Implementar `construir_svr(kernel, C, epsilon, gamma, degree)` usando `TransformedTargetRegressor` + `StandardScaler`.
3. Entrenar el baseline (`linear`, C=1) como primer run del barrido y documentar sus métricas como punto de partida; no se crea un run extra.
4. Ejecutar el barrido de 4 kernels × 4 valores de C (16 runs), registrando parámetros, RMSE, MAE, $R^2$, MAPE y **una versión del modelo** por run, con tags de kernel y C.
5. Detectar y documentar el comportamiento inestable de `sigmoid` sin ocultar resultados.
6. Ejecutar `GridSearchCV` 5-fold sobre C, epsilon, gamma y degree; registrar la búsqueda como un run adicional y el mejor modelo como versión 17.
7. Asignar el alias `champion` a la mejor versión y seleccionarla justificando la decisión con RMSE de CV frente a RMSE de test.
8. Exportar cada modelo a `resultados/modelos/` con `joblib` y generar la tabla consolidada desde `search_model_versions` + las métricas de su run.
9. Documentar hiperparámetros, semilla y versión de scikit-learn.

**Entregables:**

- Notebook secciones 4–6 ejecutando.
- ≥ 17 runs y ≥ 17 versiones visibles en `mlflow ui` (16 del barrido + 1 de optimización).
- Alias `champion` apuntando a la mejor versión, cargable con `models:/svr_ventas@champion`.
- `resultados/modelos/*.joblib` (una por versión) y tabla de resultados cruda.

**Criterio de aceptación:**

- Todas las versiones tienen los tags `kernel` y `C`, y su run tiene las métricas completas.
- El mejor modelo se puede reconstruir y cargar desde el registry sin reentrenar.
- La comparación CV vs test está explicada, no solo mostrada.

**Interacción:** depende del split de R1; entrega el modelo final a R3 y los resultados a R4 para las conclusiones. **No le corresponde:** figuras de evaluación final, función de predicción, conclusiones ni referencias.

| Actividad | Horas |
|---|---|
| Configuración de MLflow + registry | 1.5 |
| Pipeline y baseline | 2.0 |
| Barrido kernels × C | 3.0 |
| GridSearchCV y selección | 2.5 |
| Documentación y tabla | 1.0 |
| **Total** | **10.0** |

### 5.2.3 R3 — Evaluación y predicción (9 h)

**Objetivo:** medir correctamente el modelo final, comunicar los resultados con figuras claras y entregar una función de predicción usable con dos casos demostrativos.

**Tareas concretas:**

1. Implementar las métricas RMSE, MAE, $R^2$ y MAPE en una función reutilizable.
2. Evaluar el mejor modelo sobre `X_test` y construir la tabla de métricas finales.
3. Generar las figuras: predicho vs real con recta `y = x`, histograma de residuos y barras comparativas de RMSE por kernel y por C.
4. Exportar la tabla consolidada desde las **versiones registradas** a `resultados/resultados_modelos.csv` y ordenarla por RMSE.
5. Implementar `predecir_venta(nuevo_sample)` cargando `models:/svr_ventas@champion`, alineando columnas con `COLUMNAS`, rellenando categóricas faltantes con 0 y devolviendo la predicción redondeada.
6. Diseñar dos casos con productos y contextos distintos: uno de demanda alta (fin de semana/feriado, lags altos) y uno de demanda baja (entre semana, lags bajos).
7. Comparar cada predicción con un valor real cercano del dataset y explicar la diferencia.
8. Probar casos límite (campo faltante, categoría no vista) y documentar el comportamiento.
9. Exportar el notebook a HTML y revisar que tablas y figuras se vean bien.

**Entregables:**

- Notebook secciones 7–9 ejecutando.
- `resultados/resultados_modelos.csv`.
- Figuras `predicho_vs_real.png`, `residuos.png`, `comparacion_kernels.png`.
- Función `predecir_venta()` con dos salidas documentadas.
- `src/svr_ventas.html` generado.

**Criterio de aceptación:**

- Las métricas del mejor modelo coinciden con las registradas en su versión de MLflow.
- La función responde para los dos casos y no falla con campos opcionales ausentes.
- El HTML se abre y muestra todas las secciones sin errores.

**Interacción:** recibe el modelo de R2; entrega los resultados y las figuras a R4 para las conclusiones. **No le corresponde:** elegir hiperparámetros ni redactar conclusiones ni referencias.

| Actividad | Horas |
|---|---|
| Métricas y tabla final | 1.5 |
| Figuras de evaluación | 2.0 |
| Consolidado desde MLflow | 1.0 |
| Función de predicción | 2.0 |
| Dos casos y pruebas límite | 1.0 |
| Exportación HTML y revisión | 1.5 |
| **Total** | **9.0** |

### 5.2.4 R4 — Conclusiones y documentación (6 h)

**Objetivo:** cerrar el trabajo con conclusiones basadas en métricas, referencias en APA 7 y una revisión final contra el alcance acordado. **No** incluye justificar predicciones ni usar SHAP/PDP/permutation importance.

**Tareas concretas:**

1. Revisar las métricas, la tabla consolidada y las figuras que entregan R2 y R3.
2. Redactar las conclusiones con cifras: mejor kernel/C, comparación con el baseline y con el LSTM de la práctica 001, y limitaciones del dataset.
3. Construir la lista de referencias en APA 7 y citarlas en el texto.
4. Verificar la coherencia entre objetivos, resultados y conclusiones.
5. Revisar el notebook contra los ítems 3.1, 3.2 y 3.4 del alcance.
6. Coordinar con R3 la exportación HTML y revisar que se vea correctamente.
7. Preparar el guion de sustentación y repartir los bloques entre los cuatro integrantes.

**Entregables:**

- Notebook secciones 10–11 ejecutando.
- Conclusiones con métricas y referencias APA 7.
- Checklist de rúbrica marcado.
- Guion de sustentación.

**Criterio de aceptación:**

- Las conclusiones citan números concretos (RMSE, $R^2$, kernel, C).
- Las referencias están en APA 7 y aparecen citadas en el texto.
- La revisión final no deja ítems del alcance sin evidencia.
- No aparece ninguna sección de SHAP, PDP/ICE ni permutation importance.

**Interacción:** depende de los resultados de R2 y de las figuras de R3. **No le corresponde:** modificar el pipeline, recalcular métricas, cambiar el split ni implementar la función de predicción.

| Actividad | Horas |
|---|---|
| Revisión de resultados y figuras | 1.0 |
| Redacción de conclusiones | 2.0 |
| Referencias APA | 1.0 |

| Actividad | Horas |
|---|---|
| Revisión de rúbrica y HTML | 1.0 |
| Guion de sustentación | 1.0 |
| **Total** | **6.0** |

### 5.3 Matriz de responsabilidades

`E` = ejecuta, `A` = aprueba, `C` = consultado, `I` = informado.

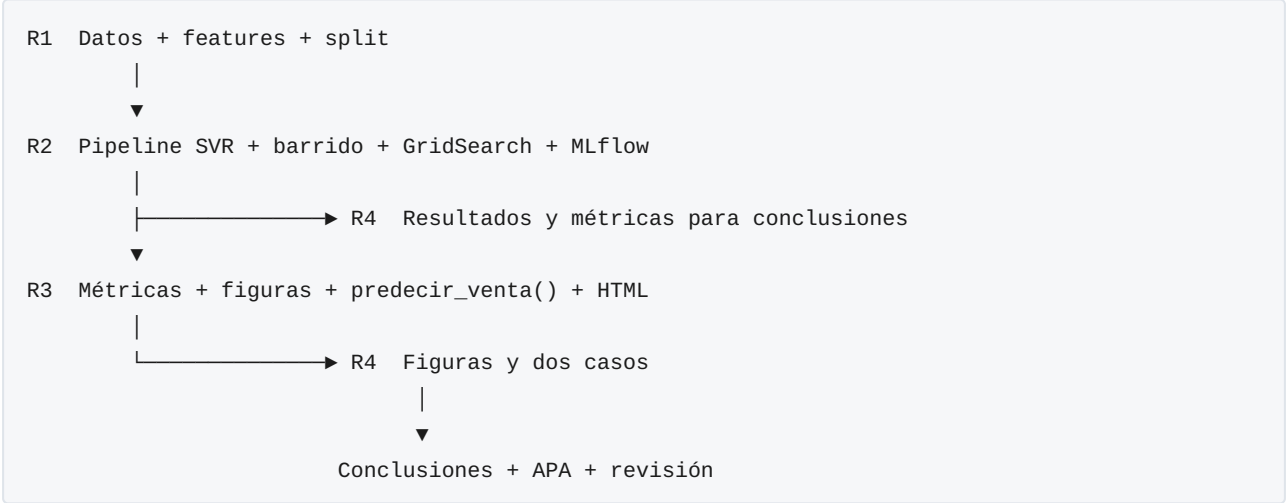
| Tarea | R1 | R2 | R3 | R4 |
|---|---|---|---|---|
| Estructura del repo y entorno | **E** | C | I | I |
| Carga, EDA y features | **E** | C | I | I |
| Split temporal y control de fuga | **E** | A | C | C |
| Pipeline SVR y escalado del target | C | **E** | C | I |
| Barrido kernels × C | I | **E** | C | I |
| GridSearchCV | I | **E** | C | I |
| Métricas y figuras de evaluación | I | C | **E** | C |
| Tabla consolidada de versiones | I | **E** | C | I |
| `predecir_venta()` y dos casos | I | C | **E** | C |
| Conclusiones | C | C | C | **E** |
| Referencias APA | I | I | I | **E** |
| Exportación HTML y revisión de rúbrica | I | I | **E** | A |

### 5.4 Regla de edición del notebook

El notebook es un archivo único y cuatro personas no pueden editarlo a la vez.

1. Cada integrante tiene su rama: `feat/datos`, `feat/modelos`, `feat/evaluacion`, `feat/documentacion`.
2. Solo se edita la sección asignada.
3. Antes de editar: `git pull --rebase origin main`.
4. Al terminar: commit pequeño y push; el integrador (R1) hace el merge.
5. Si hay conflicto, se resuelve en pareja (autor + integrador) y se vuelve a ejecutar el notebook completo.
6. Prohibido dejar celdas con errores o salidas obsoletas en `main`.

### 5.5 Flujo de dependencias entre roles



Reglas del flujo:

- R1 no entrega hasta que R2 confirme que no hay fuga de información.
- R2 no entrega hasta que la tabla de MLflow esté completa.
- R3 no entrega hasta que la función funcione con los dos casos.
- R4 no entrega hasta que las conclusiones tengan cifras y la rúbrica esté marcada.
- Ningún rol modifica el trabajo de otro sin avisar y sin volver a ejecutar el notebook.

---

## 6. Cronograma

### 6.1 Sesiones

| Sesión | Duración | Objetivo | Actividades | Responsable | Entregable |
|---|---|---|---|---|---|
| **S1** | 2 h | Base reproducible | Kickoff, repo, entorno, carga, EDA, features, split | R1 | Notebook secciones 0–3 ejecutando |
| **S2** | 2 h | Primer modelo | Pipeline SVR, baseline, configuración de MLflow, 4 runs | R2 | Sección 4–5 con 4 runs |
| **S3** | 2 h | Optimización | 16 runs, `GridSearchCV`, tabla consolidada, figuras | R2 + R3 | Secciones 5–8 completas |
| **S4** | 2 h | Predicción | Métricas finales, `predecir_venta()`, dos casos | R3 | Secciones 7 y 9 |
| **S5** | 2 h | Conclusiones y cierre | Conclusiones, APA, exportación HTML, revisión de rúbrica | R4 + todos | Notebook completo + HTML |
| **S6** | 1 h | Buffer | Correcciones, ensayo de sustentación | Todos | Versión final |

### 6.2 Hitos

| Hito | Fecha | Criterio de cumplimiento |
|---|---|---|
| **M1** | Fin S1 | El notebook carga datos y genera `X_train`, `X_test` sin errores |
| **M2** | Fin S3 | 17 runs y 17 versiones visibles en `mlflow ui`, alias `champion` y tabla `resultados_modelos.csv` |
| **M3** | Fin S4 | `predecir_venta()` devuelve valores para dos casos distintos |
| **M4** | Fin S5 | Notebook ejecutado de inicio a fin, HTML generado, rúbrica completa |

### 6.3 Trabajo entre sesiones

| Entre | Tarea | Responsable | Tiempo |
|---|---|---|---|
| S1–S2 | Revisar dokumentación de SVR y preparar el pipeline | R2 | 2 h |
| S2–S3 | Preparar funciones de métricas y figuras | R3 | 2 h |
| S3–S4 | Revisar resultados y preparar el borrador de conclusiones | R4 | 2 h |
| S4–S5 | Redactar conclusiones y referencias | R4 | 2 h |
| S5–S6 | Ensayo individual de la sustentación | Todos | 1 h |

---

## 7. Guía técnica paso a paso

### 7.1 Preparar el entorno

Desde `practica-002`:

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
python -m pip check
python -m ipykernel install --user --name svr-ventas --display-name "Python (svr-ventas)"
```

Las versiones exactas, la verificación y los problemas frecuentes están en `docs/entorno.md`.

Verificación rápida:

```
python -c "import pandas, sklearn, mlflow; print(pandas.__version__, sklearn.__version__,
mlflow.__version__)"
```

Salida esperada: `3.0.6 1.9.1 3.18.0`.

### 7.2 Estructura de carpetas

```
practica-002/
├── datos/
│   ├── ventas_features.csv        # dataset principal
│   ├── sales_history.csv          # ventas crudas
│   └── inventory.csv              # catálogo de productos
├── docs/
│   ├── entorno.md
│   ├── guia.md
│   ├── guia.docx
│   ├── plan_desarrollo_svr_4_personas.md
│   └── plan_desarrollo_svr_4_personas.pdf
├── src/
│   ├── <integrante>/
│   │   └── practica-002-vN/       # entrega por versión
│   └── svr_ventas.ipynb           # integración final
├── resultados/
│   ├── resultados_modelos.csv     # una fila por versión registrada
│   ├── modelos/                   # un .joblib por versión
│   └── figuras/
├── mlflow.db                      # registry SQLite (ignorado en Git)
├── mlartifacts/                   # artefactos de los runs (ignorado en Git)
├── requirements.txt
├── requirements-lock.txt
├── .python-version
├── datos.sha256
└── README.md
```

Verificar los datos:

```
mkdir -p resultados/figuras resultados/modelos
sha256sum -c datos.sha256
```

### 7.3 Carga, features y target

```
import pandas as pd

df = pd.read_csv("datos/ventas_features.csv", parse_dates=["fecha_venta"])
df = df.sort_values(["fecha_venta", "product_id"]).reset_index(drop=True)

NUM = [
    "mes", "dia_del_mes", "dia_semana", "es_fin_de_semana", "es_feriado",
    "semana_del_periodo", "ventas_dia_anterior", "ventas_hace_7_dias",
    "promedio_movil_7d", "stock_en_momento", "dias_para_expirar",
]
CAT = ["categoria", "product_id"]
TARGET = "cantidad_vendida"

df = df.dropna(subset=NUM + [TARGET]).reset_index(drop=True)

X = pd.get_dummies(df[NUM + CAT], columns=CAT, drop_first=True, dtype=float)
y = df[TARGET]

print(df.shape, X.shape)
```

### 7.4 Split temporal

```
corte = df["fecha_venta"].quantile(0.8)
train = df["fecha_venta"] <= corte
test = ~train

X_train, y_train = X[train], y[train]
X_test, y_test = X[test], y[test]

print("Corte:", corte.date())
print("Train:", train.sum(), "Test:", test.sum())
```

Guardar `COLUMNAS = X_train.columns.tolist()` porque la función de predicción lo necesita.

### 7.5 Pipeline SVR con target escalado

```
from sklearn.compose import TransformedTargetRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

def construir_svr(kernel="rbf", C=1.0, epsilon=0.1, gamma="scale", degree=3):
    return TransformedTargetRegressor(
        regressor=make_pipeline(
            StandardScaler(),
            SVR(kernel=kernel, C=C, epsilon=epsilon, gamma=gamma, degree=degree),
        ),
        transformer=StandardScaler(),
    )
```

> ¿Por qué escalar `y`? `epsilon` define una banda de tolerancia en unidades del target. Con ventas de 67–411, `epsilon=0.1` obliga al modelo a ajustar casi exactamente cada punto y sobreajusta. Escalar el target hace que `epsilon` y `C` sean interpretables y comparables entre corridas.

### 7.6 Barrido kernels × C con MLflow

```
import joblib
import numpy as np
import mlflow
import mlflow.sklearn
from mlflow import MlflowClient
from pathlib import Path
from sklearn.metrics import (
    mean_squared_error, mean_absolute_error, r2_score,
    mean_absolute_percentage_error,
)

mlflow.set_tracking_uri("sqlite:///mlflow.db")   # el registry exige backend SQL
mlflow.set_experiment("svr_ventas")

cliente = MlflowClient()
NOMBRE = "svr_ventas"
Path("resultados/modelos").mkdir(parents=True, exist_ok=True)

for kernel in ["linear", "rbf", "poly", "sigmoid"]:
    for C in [0.1, 1, 10, 100]:
        with mlflow.start_run(run_name=f"{kernel}_C{C}"):
            modelo = construir_svr(kernel=kernel, C=C)
            modelo.fit(X_train, y_train)
            pred = modelo.predict(X_test)

            rmse = float(np.sqrt(mean_squared_error(y_test, pred)))
            mae = float(mean_absolute_error(y_test, pred))
            r2 = float(r2_score(y_test, pred))
            mape = float(mean_absolute_percentage_error(y_test, pred) * 100)

            mlflow.log_params({
                "kernel": kernel, "C": C, "epsilon": 0.1, "gamma": "scale",
                "split": "temporal", "target": TARGET,
            })
            mlflow.log_metrics({"rmse": rmse, "mae": mae, "r2": r2, "mape": mape})

            # Una versión del modelo por run
            info = mlflow.sklearn.log_model(modelo, name="modelo")
            version = mlflow.register_model(info.model_uri, NOMBRE)
            cliente.set_model_version_tag(NOMBRE, version.version, "kernel", kernel)
            cliente.set_model_version_tag(NOMBRE, version.version, "C", str(C))

            # Copia portable para revisión sin MLflow
            joblib.dump(modelo, f"resultados/modelos/{kernel}_C{C}.joblib")
```

> Cada iteración crea un run **y** una versión del mismo modelo registrado `svr_ventas`. Las versiones son v1… v16; la etiqueta (`kernel`, `C`) permite identificarlas sin abrir el run. `register_model` devuelve el número de versión, que es lo que se etiqueta.

> `sigmoid` puede producir valores extremos. Es un resultado esperado y sirve para la discusión: no todos los kernels son adecuados para datos tabulares.

> **El registry no se limpia en re-ejecuciones** (decisión del equipo). El notebook no borra el modelo registrado ni los runs al empezar: re-ejecutar agrega 17 versiones nuevas (v18–v34 en la segunda corrida) y el alias `champion` pasa a la nueva versión del `GridSearchCV`, que es justamente el propósito del alias. Por eso los `assert` de versiones son **relativos** y no números absolutos: se captura `version_inicial = len(cliente.search_model_versions(f"name='{NOMBRE}'"))` antes del barrido y se comprueba `version == version_inicial + 17` en el `GridSearchCV`. La única excepción es el run de prueba de la configuración (tarea 1), que se borra a sí mismo porque no es un entrenamiento.

### 7.7 Optimización con GridSearchCV

```
from sklearn.model_selection import GridSearchCV, KFold

param_grid = [
    {
        "regressor__svr__kernel": ["linear"],
        "regressor__svr__C": [0.1, 1, 10, 100],
        "regressor__svr__epsilon": [0.01, 0.1, 0.5],
    },
    {
        "regressor__svr__kernel": ["rbf"],
        "regressor__svr__C": [0.1, 1, 10, 100],
        "regressor__svr__epsilon": [0.01, 0.1, 0.5],
        "regressor__svr__gamma": ["scale", 0.01, 0.1],
    },
    {
        "regressor__svr__kernel": ["poly"],
        "regressor__svr__C": [0.1, 1, 10],
        "regressor__svr__epsilon": [0.1, 0.5],
        "regressor__svr__gamma": ["scale", 0.1],
        "regressor__svr__degree": [2, 3],
    },
]

gs = GridSearchCV(
    construir_svr(),
    param_grid,
    cv=KFold(5, shuffle=True, random_state=42),
    scoring="neg_root_mean_squared_error",
    n_jobs=-1,
)
gs.fit(X_train, y_train)

mejor_modelo = gs.best_estimator_
print("Mejores parámetros:", gs.best_params_)
print("RMSE CV:", -gs.best_score_)
```

Registrar la búsqueda como un run y el mejor modelo como versión 17, con el alias `champion`:

```
with mlflow.start_run(run_name="gridsearch"):
    mlflow.log_params(gs.best_params_)
    mlflow.log_metrics({"rmse_cv": float(-gs.best_score_)})

    info = mlflow.sklearn.log_model(mejor_modelo, name="modelo")
    version = mlflow.register_model(info.model_uri, NOMBRE)
    cliente.set_registered_model_alias(NOMBRE, "champion", version.version)

joblib.dump(mejor_modelo, "resultados/modelos/svr_mejor.joblib")
```

> El alias apunta siempre a la versión vigente: `predecir_venta()` carga `models:/svr_ventas@champion` y no hay que tocar código cuando cambie el mejor modelo.

### 7.8 Consolidar resultados desde MLflow

```
filas = []
for v in cliente.search_model_versions(f"name='{NOMBRE}'"):
    metricas = cliente.get_run(v.run_id).data.metrics
    filas.append({
        "version": int(v.version),
        "kernel": v.tags.get("kernel", ""),
        "C": float(v.tags.get("C", "nan")),
        "RMSE": metricas.get("rmse"),
        "MAE": metricas.get("mae"),
        "R2": metricas.get("r2"),
        "MAPE": metricas.get("mape"),
        "run": v.run_id,
    })

tabla = pd.DataFrame(filas).sort_values("RMSE").reset_index(drop=True)
tabla.to_csv("resultados/resultados_modelos.csv", index=False)
tabla
```

Esta tabla **es** la "lista de resultados de todos los modelos" que pide la guía, y ahora sale de las **versiones**, no de los runs sueltos.

> La tabla de versiones del UI de MLflow no muestra métricas: solo número, alias, estado y run. Por eso hay que unir cada versión con su run ( `get_run(v.run_id)` ) para comparar RMSE/MAE. La columna `version` permite volver a cargar cualquier modelo con `models:/svr_ventas/<version>` .

> Si el notebook se re-ejecutó y el registry acumula versiones de corridas anteriores, filtra `search_model_versions` a los runs de la ejecución corriente para que la tabla quede con las 17 filas de esa corrida.

Incluir una figura de barras: eje X = C, eje Y = RMSE, una barra por kernel.

### 7.9 Evaluación y figuras

- Tabla de métricas del mejor modelo.
- Scatter **predicho vs real** con la recta `y = x` .
- Histograma de **residuos** ( `y_test - pred` ).
- Barras comparativas de RMSE por kernel y por C.

Todas las figuras se guardan en `resultados/figuras/` con `dpi=150` y nombre descriptivo, por ejemplo `comparacion_kernels.png` .

### 7.10 Función de predicción para un sample nuevo

```
# La función usa la versión vigente del registry, no la variable en memoria:
# así se demuestra que el modelo quedó guardado y se puede reutilizar después.
mejor_modelo = mlflow.sklearn.load_model("models:/svr_ventas@champion")

COLUMNAS = X_train.columns.tolist()

def predecir_venta(nuevo_sample: dict) -> dict:
    """Predice la cantidad vendida para un día y producto nuevos."""
    fila = pd.DataFrame([nuevo_sample])
    fila = fila.drop(columns=["fecha_venta"], errors="ignore")
    fila = pd.get_dummies(fila, dtype=float).reindex(columns=COLUMNAS, fill_value=0.0)

    prediccion = float(mejor_modelo.predict(fila)[0])
    return {
        "venta_estimada": round(prediccion, 1),
        "unidad": "unidades/día",
    }
```

**Caso 1 — demanda alta** (fin de semana, feriado, lag alto):

```
caso_alto = {
    "mes": 2, "dia_del_mes": 12, "dia_semana": 6,
    "es_fin_de_semana": 1, "es_feriado": 1, "semana_del_periodo": 7,
    "ventas_dia_anterior": 220, "ventas_hace_7_dias": 205,
    "promedio_movil_7d": 210, "stock_en_momento": 700,
    "dias_para_expirar": 150,
    "categoria": "Bebidas", "product_id": "P0008",
}
print(predecir_venta(caso_alto))
```

**Caso 2 — demanda baja** (entre semana, sin feriado, lag bajo):

```
caso_bajo = {
    "mes": 2, "dia_del_mes": 15, "dia_semana": 2,
    "es_fin_de_semana": 0, "es_feriado": 0, "semana_del_periodo": 7,
    "ventas_dia_anterior": 75, "ventas_hace_7_dias": 80,
    "promedio_movil_7d": 78, "stock_en_momento": 300,
    "dias_para_expirar": 120,
    "categoria": "Alimentos", "product_id": "P0004",
}
print(predecir_venta(caso_bajo))
```

Además de la predicción, mostrar el **valor real** de un registro parecido del dataset para comparar. Documentar que los dos samples pertenecen a productos y contextos distintos.

### 7.11 Cierre: HTML y MLflow UI

```
jupyter nbconvert --to html src/svr_ventas.ipynb
mlflow ui --backend-store-uri sqlite:///mlflow.db
# abrir http://127.0.0.1:5000 → Experiments (runs) y Models → svr_ventas (versiones)
```

Antes de exportar: reiniciar kernel y ejecutar todo (`Restart & Run All`) para que el HTML no tenga errores ni salidas viejas.

---

## 8. Convenciones de trabajo

### 8.1 Nombres

| Elemento | Convención | Ejemplo |
|---|---|---|
| Notebook | snake_case | `svr_ventas.ipynb` |
| Experimento MLflow | snake_case | `svr_ventas` |
| Run MLflow | `kernel_Cvalor` | `rbf_C10` |
| Figura | descriptiva | `comparacion_kernels.png` |
| Rama | `feat/...` | `feat/modelos` |
| Commit | verbo + objeto | `agrega barrido de kernels con MLflow` |

### 8.2 Calidad del notebook

- Una celda Markdown antes de cada bloque de código.
- Sin código muerto ni celdas duplicadas.
- Sin `print` de depuración en la versión final.
- Semilla fija en todo lo aleatorio.
- El notebook debe correr de inicio a fin en menos de 5 minutos.
- Gráficos con título, ejes rotulados y unidades.

### 8.3 Git

```
git checkout -b feat/modelos
git add src/svr_ventas.ipynb
git commit -m "agrega barrido de kernels y C con MLflow"
git pull --rebase origin main
git push origin feat/modelos
```

- Nunca commitear `.venv/`, `mlflow.db`, `mlartifacts/` ni `__pycache__/`. El `.gitignore` debe incluirlos; los modelos para revisión se versionan como `.joblib` en `resultados/modelos/`.
- Los artefactos de MLflow se conservan localmente para la demostración; si el docente pide el repositorio, se documenta cómo regenerarlos.

### 8.4 Comunicación

- Un canal único del equipo (WhatsApp/Discord) para dudas y avisos.
- Reunión de 10 minutos al inicio de cada sesión: qué se hizo, qué sigue, bloqueos.
- Si alguien no puede asistir, avisa con 24 h y deja su parte documentada.
- Las decisiones técnicas se registran en la sección 3 de este documento, no en el chat.

---

## 9. Criterios de aceptación (checklist de rúbrica)

Marcar antes de la entrega:

- ☐ **3.1** Se entrenaron los 4 kernels y al menos 4 valores de C.
- ☐ **3.1** `X` tiene 25 columnas (`categoria` + `product_id`) y la sección 3.6 explica la duplicación.

- □ **3.1** Los 17 runs aparecen en `mlflow ui` con parámetros y métricas.
- □ **3.1** Las 17 versiones aparecen en **Models → svr_ventas** con sus tags de kernel y C.
- □ **3.1** El alias `champion` apunta a la mejor y `models:/svr_ventas@champion` carga sin error.
- □ **3.1** Existe una tabla comparativa ( `resultados_modelos.csv` ) y figuras que resaltan el mejor modelo.
- □ **3.1** `GridSearchCV` documentado con los mejores hiperparámetros.
- □ **3.1** Métricas reportadas: RMSE, MAE, $R^2$ y MAPE.
- □ **3.2** `predecir_venta()` definida y ejecutada con dos casos distintos.
- □ **3.2** Se muestra la predicción y se compara con un valor real.
- □ **3.4** Conclusiones con métricas concretas y comparación con la práctica 001.
- □ **3.4** Referencias en APA 7, citadas en el texto.
- □ No se incluye justificación de predicciones con SHAP, PDP/ICE ni permutation importance (fuera de alcance).
- □ Notebook ejecutado de inicio a fin sin errores.
- □ `src/svr_ventas.html` generado y revisado.
- □ Repositorio ordenado y `.gitignore` correcto.
- □ Todos los integrantes pueden explicar cualquier sección.

---

## 10. Riesgos y mitigación

| Riesgo | Probabilidad | Impacto | Mitigación |
|---|---|---|---|
| Conflictos al editar el notebook | Alta | Medio | Ramas por persona, secciones asignadas, integrador único |
| Kernel `sigmoid` inestable | Alta | Bajo | Documentarlo como resultado esperado; limitar C |
| Métricas distintas entre integrantes | Media | Medio | Semilla fija, split congelado, mismos datos |
| Fuga de información en features | Media | Alto | Revisión cruzada R1–R2 antes de modelar |
| Split temporal con solo ~100 filas de test | Alta | Medio | Reportar también CV 5-fold; no sobreinterpretar decimales |
| Registry no disponible con `file:./mlruns` | Media | Alto | Usar `sqlite:///mlflow.db` desde la sección 4 |
| El `.db` no viaja en el repo | Media | Medio | Exportar `resultados_modelos.csv` y los `.joblib` de `resultados/modelos/` |
| Pérdida de una sesión | Media | Medio | Sesión buffer S6 |
| Un integrante se atrasa | Media | Alto | Tareas independientes y revisión por pares |

---

## 11. Plan de sustentación

Duración total: 12–15 minutos + preguntas. Un bloque por integrante.

| Bloque | Minutos | Contenido | Responsable |
|---|---|---|---|
| 1. Contexto | 3 | Dataset, variable objetivo, por qué SVR, split y control de fuga | R1 |
| 2. Modelo y optimización | 4 | Pipeline, kernels, C, versiones en MLflow, GridSearch, tabla de resultados | R2 |
| 3. Evaluación y predicción | 4 | RMSE/MAE/$R^2$, figuras, `predecir_venta()` y los dos casos | R3 |
| 4. Conclusiones y referencias | 3 | Hallazgos, limitaciones y referencias APA | R4 |

### Preguntas probables y respuestas cortas

- **¿Por qué SVR y no clasificación?** Porque el target `cantidad_vendida` es continuo y la guía permite regresión; inventar clases perdería información.
- **¿Por qué escalar el target?** Porque `epsilon` y `C` dependen de la escala; sin escalar, el modelo sobreajusta.
- **¿Por qué split temporal?** Para evitar que días consecutivos queden en train y test, lo que inflaría las métricas.
- **¿Por qué `sigmoid` falla?** No es un kernel adecuado para este tipo de datos tabulares; produce predicciones extremas.
- **¿Por qué se registran versiones y no solo runs?** Porque la práctica compara variantes del mismo modelo (kernel y C). El registry les da un número estable, las etiqueta y permite cargar cualquiera con `models:/svr_ventas/<version>`; los runs guardan las métricas de cada una.
- **¿Por qué no se usa SHAP?** Ese punto de justificación de predicciones quedó excluido del alcance acordado; la práctica se centra en optimización, predicción y conclusiones.
- **¿El modelo sirve para producción?** No tal cual: hay 60 días y 10 productos; se necesita más histórico y validación temporal más larga.

---

## 12. Referencias (APA 7)


Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*(3), 273–297. https://doi.org/10.1007/BF00994018

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Raschka, S. (2017). *Python machine learning* (2.ª ed.). Packt Publishing.

Smola, A. J., & Schölkopf, B. (2004). A tutorial on support vector regression. *Statistics and Computing, 14*(3), 199–222. https://doi.org/10.1023/B:STCO.0000035301.49549.88

Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nystrom, T., Ogilvie, P., Parkhe, M., Plawinski, M., & Zumar, C. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin, 41*(4), 39–45.


---

## 13. Anexos

### A. Plantilla de tabla de resultados

| Versión | Run | Kernel | C | Epsilon | Gamma | RMSE | MAE | $R^2$ | MAPE |
|---|---|---|---|---|---|---|---|---|---|
| 1 | linear_C0.1 | linear | 0.1 | 0.1 | — | | | | |
| 2 | linear_C1 | linear | 1 | 0.1 | — | | | | |
| 11 | rbf_C10 | rbf | 10 | 0.1 | scale | | | | |
| 12 | poly_C10 | poly | 10 | 0.1 | scale | | | | |
| 17 | gridsearch | rbf | 10 | 0.01 | 0.01 | | | | |

### B. Checklist de revisión final

1. Reiniciar kernel y ejecutar todo el notebook.
2. Verificar que no haya warnings graves ni celdas con error.
3. Revisar que los 17 runs y las 17 versiones estén en MLflow, con el alias `champion` asignado a la mejor.
4. Abrir `resultados_modelos.csv` y confirmar el orden por RMSE.
5. Revisar que cada figura esté citada e interpretada en Markdown.

6. Confirmar que los dos casos de predicción usan productos y contextos distintos.
7. Revisar ortografía y formato APA.
8. Exportar HTML y abrirlo en el navegador.
9. Ensayar la sustentación completa una vez.

### C. Comandos útiles

```
# Activar entorno
source .venv/bin/activate

# Ejecutar el notebook desde la terminal
jupyter nbconvert --to notebook --execute --inplace src/svr_ventas.ipynb

# Exportar a HTML
jupyter nbconvert --to html src/svr_ventas.ipynb

# Ver experimentos y versiones
mlflow ui --backend-store-uri sqlite:///mlflow.db

# Revisar estado del repositorio
git status
git log --oneline --graph --decorate
```

### D. Glosario mínimo

- **SVR:** Support Vector Regression, SVM adaptada a variables continuas.
- **Kernel:** función que mapea los datos a un espacio de mayor dimensión.
- **C:** penalización por error; valores altos ajustan más y pueden sobreajustar.
- **Epsilon:** ancho de la banda de tolerancia del SVR.
- **Gamma:** alcance de influencia de cada punto en el kernel RBF.
- **Fuga de información:** usar variables que contienen información del target o del futuro.
- **Model Registry:** catálogo de modelos de MLflow; cada entrenamiento guardado se convierte en una versión numerada del modelo registrado.
- **Versión:** una fila del registry ( `svr_ventas` v1, v2…), ligada a su run y a sus métricas.
- **Alias:** nombre mutable que apunta a una versión; `champion` marca la vigente sin cambiar el código que la carga.

---

**Nota:** este documento es el plan de desarrollo. La implementación del notebook se realiza en el agente de construcción; aquí solo se definen decisiones, responsables, cronograma y criterios de aceptación.